Tópicos Avançados de Aprendizagem Máquina

Trabalho Prático de Processamento de Linguagem Natural

Avaliação Experimental de Técnicas de Vetorização de Texto em Classificação Binária de Mensagens SMS

Ana Estreia

19/06/2026

1. Introdução

1.1. Contexto e Motivação

A classificação automática de texto é uma das tarefas centrais do Processamento de Linguagem Natural (PLN), com aplicações práticas como filtragem de *spam*, análise de sentimentos, categorização de notícias e deteção de conteúdo ofensivo. Neste trabalho aborda-se um problema concreto e amplamente estudado na literatura: a classificação binária de mensagens SMS como *spam* (mensagens indesejadas) ou *ham* (mensagens legítimas). Este cenário é particularmente interessante por combinar mensagens curtas e informais com um desequilíbrio de classes realista, o que levanta desafios relevantes tanto a nível de representação textual como de escolha do algoritmo de classificação.

O objetivo principal deste trabalho é compreender de que forma diferentes opções de pré-processamento, de representação de texto e de algoritmo de classificação influenciam o desempenho de um sistema real de classificação binária de texto.

1.2. Objetivos

- Avaliar o impacto do pré-processamento do texto, comparando uma versão com lematização e remoção de *stop words* com uma versão de limpeza básica (apenas conversão para minúsculas e remoção de pontuação);
- Comparar diferentes representações de texto: *Bag-of-Words*, TF-IDF, Word2Vec e Doc2Vec;
- Comparar diferentes algoritmos de classificação: Regressão Logística, SVM e *Random Forest*;
- Avaliar o efeito de técnicas de reamostragem (ex.: SMOTE e *RandomUnderSampler*) na mitigação do desequilíbrio entre classes, face a um *baseline* desbalanceado;
- Implementar um pipeline modular e reprodutível com scikit-learn, com otimização de hiperparâmetros através de *GridSearchCV* e validação cruzada;
- Analisar criticamente os erros de classificação, identificando padrões nas mensagens incorretamente classificadas.

1.3. Estrutura do Relatório

O presente relatório está organizado em cinco secções principais. A secção 2 descreve a metodologia seguida, incluindo o corpus, as estratégias de pré-processamento, as representações de texto e os algoritmos de classificação avaliados, bem como o procedimento experimental. A secção 3 apresenta os resultados experimentais obtidos, sintetizados numa tabela comparativa e em visualizações de apoio. A secção 4 discute criticamente esses resultados, relacionando-os com as escolhas metodológicas e identificando padrões de erro. Por fim, a secção 5 sintetiza as principais conclusões do trabalho e aponta possíveis direções de trabalho futuro.

2. Metodologia

2.1. Corpus

Foi utilizado o *dataset* SMS *Spam Collection*, disponível no repositório UCI Machine Learning, composto por mensagens SMS anotadas manualmente como *spam* ou *ham*. As principais características do corpus estão resumidas na Tabela 1.

Característica	Descrição
Fonte	UCI Machine Learning Repository - SMS *Spam Collection*
Classes	*spam* (mensagens indesejadas) e *ham* (mensagens legítimas)
Dimensão	5.572 mensagens
Distribuição	Desbalanceada: 86,59% *ham* (4.825) / 13,41% *spam* (747)
Divisão treino/teste	80/20 estratificado: 4.457 treino / 1.115 teste

Tabela 1. Características do corpus SMS Spam Collection.

2.2. Pré-processamento de Texto (Tarefa T2)

Foram comparadas duas configurações de pré-processamento:

- **Config A (*raw*):** conversão para minúsculas e remoção de pontuação. Configuração mínima que preserva o vocabulário original.
- **Config B (lemm_stop):** Config A acrescida de remoção de *stop words* e lematização com `WordNetLemmatizer`. Reduz o vocabulário a radicais semânticos e elimina palavras funcionais de baixo valor discriminativo.

A função `clean_text` partilhada pelas duas configurações remove todos os caracteres não-alfabéticos, incluindo dígitos, símbolos monetários e pontuação.

2.3. Representações de Texto (Tarefa T3)

Foram avaliadas quatro representações:

- **Bag-of-Words (BoW)**: vetorização por contagem de ocorrências de termos (`CountVectorizer`).
- **TF-IDF:** vetorização por frequência de termos ponderada pela frequência inversa de documentos (`TfidfVectorizer`).
- **Word2Vec:** *embeddings* densos, treinados no corpus de treino com `gensim.Word2Vec`. A representação de cada mensagem é a média dos vetores das palavras que a compõem.
- **Doc2Vec:** *embeddings* de parágrafo, treinados com `gensim.Doc2Vec`. Aprende uma representação global para cada documento, sem necessitar de agregação explícita.

Para BoW e TF-IDF foram testadas as duas configurações de pré-processamento (Config A e Config B). Para Word2Vec e Doc2Vec, foi usada apenas a Config B, uma vez que a lematização é a preparação mais adequada para modelos de *embeddings*.

2.4. Algoritmos de Classificação (Tarefa T3)

Foram avaliados três algoritmos:

- **Regressão Logística (LR)**
  Hiperparâmetros testados: `C ∈ {0.1, 1, 10}`.
- **SVM (*Support Vector Machine*)**
  Hiperparâmetros testados: `kernel ∈ {linear, rbf}, C ∈ {0.1, 1, 10}`.
- ***Random Forest* (RF)**
  Hiperparâmetros testados: `n_estimators ∈ {100, 200}, max_depth ∈ {None, 10, 20}`.

2.5. Técnicas de Balanceamento de Classes(Tarefa T1)

Para lidar com o desequilíbrio de classes (13,41% *spam*), foram testadas três condições:

- **Sem reamostragem (*baseline*):** dados de treino originais, com a proporção natural das classes.
- **SMOTE (*Synthetic Minority Over-sampling Technique*):** *oversampling* da classe minoritária por interpolação de exemplos reais, gerando exemplos sintéticos de *spam* no espaço de *features*.
- ***RandomUnderSampler*:** *undersampling* aleatório da classe maioritária (*ham*), reduzindo o desequilíbrio pela eliminação de exemplos.

Para evitar fuga de dados (*data leakage*), a reamostragem foi integrada no pipeline usando `imblearn.pipeline.Pipeline`, garantindo que o passo de *resampling* só atua sobre os dados de treino de cada fold da validação cruzada, nunca sobre os dados de validação ou teste.

2.6. Procedimento Experimental

O procedimento experimental seguiu os seguintes passos:

- Divisão treino/teste estratificada (80/20): 4.457 exemplos de treino, 1.115 de teste.
- Para cada combinação representação × classificador × pré-processamento × reamostragem, foi construído um *Pipeline* scikit-learn (ou imblearn) e otimizado por GridSearchCV com 5-*fold cross-validation*, minimizando pelo F1-*score* (`refit='f1'`).
- Foram registados, para cada experimento: melhores hiperparâmetros, F1 e accuracy em validação cruzada (CV) e em teste, matriz de confusão e tempo de execução.
- Reprodutibilidade garantida com `random_state=42` e `np.random.seed(42)` em todas as etapas estocásticas.
- Ambiente de execução: Google Colab (CPU).

No total foram realizados 27 experimentos, correspondendo a 570 combinações de hiperparâmetros avaliadas (510 combinações únicas, descontando as 3 repetições entre T1 e T2), totalizando 2.850 modelos treinados considerando os 5 *folds* de validação cruzada.

No total foram realizados 27 experimentos, avaliando 450 combinações de hiperparâmetros (90 por cada uma das 5 *folds* de CV)

3. Resultados Experimentais

3.1. Tabela Comparativa

Representação	Classificador	Pré-proc.	Balanceamento	Acc CV	F1 CV	Acc Teste	F1 Teste	Combinações	Tempo (s)
BoW	LR	B (lemm)	Nenhum	0.980	0.921	0.985	0.940	18	12
TF-IDF	SVM (linear)	A (raw)	Nenhum	0.984	0.938	0.984	0.937	24	105
TF-IDF	SVM (linear)	A (raw)	Nenhum	0.984	0.938	0.984	0.937	24	99
BoW	LR	A (raw)	Nenhum	0.981	0.926	0.984	0.936	18	16
BoW	SVM (linear)	A (raw)	Nenhum	0.980	0.922	0.982	0.931	36	134
TF-IDF	LR	B (lemm)	Nenhum	0.980	0.921	0.982	0.929	12	6
TF-IDF	LR	A (raw)	Nenhum	0.982	0.931	0.982	0.929	12	10
TF-IDF	LR	A (raw)	Nenhum	0.982	0.931	0.982	0.929	12	9
TF-IDF	RF	A (raw)	SMOTE	0.978	0.913	0.982	0.929	24	156
TF-IDF	SVM (linear)	A (raw)	SMOTE	0.981	0.928	0.981	0.928	24	199
TF-IDF	SVM (linear)	B (lemm)	Nenhum	0.982	0.928	0.981	0.926	24	99
TF-IDF	SVM (linear)	A (raw)	RUS	0.980	0.926	0.978	0.920	24	30
BoW	SVM (linear)	B (lemm)	Nenhum	0.980	0.919	0.979	0.918	36	103
BoW	RF	A (raw)	Nenhum	0.975	0.899	0.979	0.916	36	179
TF-IDF	RF	B (lemm)	Nenhum	0.976	0.901	0.978	0.908	24	132
TF-IDF	LR	A (raw)	RUS	0.971	0.899	0.974	0.907	12	8
TF-IDF	RF	A (raw)	RUS	0.973	0.902	0.973	0.903	24	68
TF-IDF	LR	A (raw)	SMOTE	0.976	0.914	0.973	0.902	12	9
BoW	RF	B (lemm)	Nenhum	0.976	0.905	0.976	0.900	36	188
Word2Vec	RF	B (lemm)	Nenhum	0.961	0.846	0.968	0.873	24	579
TF-IDF	RF	A (raw)	Nenhum	0.976	0.902	0.970	0.871	24	139
TF-IDF	RF	A (raw)	Nenhum	0.976	0.902	0.970	0.871	24	107
Doc2Vec	SVM (linear)	B (lemm)	Nenhum	0.972	0.889	0.967	0.863	12	509
Doc2Vec	RF	B (lemm)	Nenhum	0.972	0.890	0.962	0.843	12	778
Word2Vec	SVM (linear)	B (lemm)	Nenhum	0.943	0.764	0.959	0.837	24	155
Word2Vec	LR	B (lemm)	Nenhum	0.936	0.721	0.954	0.815	12	64
Doc2Vec	LR	B (lemm)	Nenhum	0.969	0.875	0.952	0.798	6	260

Tabela 2. Resultados de todos os experimentos, ordenados por F1-score no conjunto de teste. LR = Logistic Regression; SVM = Support Vector Machine; RF = Random Forest; A = Config A (raw); B = Config B (lemm+stop); RUS = RandomUnderSampler.

3.2. Impacto do Pré-processamento (Tarefa T2)

A Tabela 3 compara as duas configurações de pré-processamento para cada combinação representação × classificador. A coluna Δ F1 (A−B) indica a diferença de F1 de teste obtida pela Config A face à Config B.

Representação	Classificador	Pré-proc.	Acc CV	F1 CV	Acc Teste	F1 Teste	Δ F1 (A−B)
BoW	LR	A (raw)	0.981	0.926	0.984	0.936	-0.004
		B (lemm)	0.980	0.921	0.985	0.940	
	RF	A (raw)	0.975	0.899	0.979	0.916	0.016
		B (lemm)	0.976	0.905	0.976	0.900	
	SVM (linear)	A (raw)	0.980	0.922	0.982	0.931	0.013
		B (lemm)	0.980	0.919	0.979	0.918	
TF-IDF	LR	A (raw)	0.982	0.931	0.982	0.929	0.000
		B (lemm)	0.980	0.921	0.982	0.929	
	RF	A (raw)	0.976	0.902	0.970	0.871	-0.037
		B (lemm)	0.976	0.901	0.978	0.908	
	SVM (linear)	A (raw)	0.984	0.938	0.984	0.937	0.011
		B (lemm)	0.982	0.928	0.981	0.926	

Tabela 3. Impacto do pré-processamento (Config A vs Config B) para BoW e TF-IDF.

Em 4 das 6 combinações, a Config A (sem lematização nem *stop words*) obteve F1 igual ou superior à Config B. As exceções são BoW+LR (Config B: 0,940 vs Config A: 0,936) e TF-IDF+RF (Config B: 0,908 vs Config A: 0,871). A diferença mais acentuada ocorre em TF-IDF+RF (Δ = −0,037), sendo marginal nas restantes combinações.

3.3. Impacto do Balanceamento de Classes (Tarefa T1)

A Tabela 4 compara os três regimes de reamostragem (nenhum, SMOTE e *RandomUnderSampler*) para as combinações TF-IDF (Config A) × 3 classificadores. A coluna Δ F1 indica a diferença face ao *baseline* sem reamostragem.

Classificador	Balanceamento	Acc CV	F1 CV	Acc Teste	F1 Teste	Δ F1 vs Nenhum
LR	Nenhum	0.982	0.931	0.982	0.929	
	RUS	0.971	0.899	0.974	0.907	-0.022
	SMOTE	0.976	0.914	0.973	0.902	-0.027
RF	Nenhum	0.976	0.902	0.970	0.871	
	RUS	0.973	0.902	0.973	0.903	0.032
	SMOTE	0.978	0.913	0.982	0.929	0.058
SVM (linear)	Nenhum	0.984	0.938	0.984	0.937	
	RUS	0.980	0.926	0.978	0.920	-0.017
	SMOTE	0.981	0.928	0.981	0.928	-0.009

Tabela 4. Impacto da reamostragem (TF-IDF + Config A). Linhas sombreadas a verde marcam os baselines sem reamostragem.

3.4. Melhor Modelo e Matriz de Confusão

O melhor modelo obtido foi **TF-IDF + SVM (linear, C=1, max_df=0,75, ngram_range=(1,2)) + Config A, sem reamostragem**, com F1-score de 0,937 e accuracy de 0,984 no conjunto de teste. O desempenho em validação cruzada (F1 = 0,938) é praticamente idêntico ao de teste, indicando boa generalização. A matriz de confusão (Tabela 5) revela uma precisão muito elevada na classe *spam* (133/134 = 99,3%) mas um recall mais moderado (133/149 = 89,3%), com 16 falsos negativos e apenas 1 falso positivo.

	Previsto: ham	Previsto: spam
Real: ham	**965**	1
Real: spam	16	**133**

Tabela 5. Matriz de confusão do melhor modelo (TF-IDF + SVM linear, Config A, sem reamostragem). Conjunto de teste: 1.115 mensagens.

4. Discussão

4.1. Comparação entre Representações de Texto

As representações BoW e TF-IDF dominam consistentemente os resultados , superando amplamente Word2Vec e Doc2Vec. Este resultado, à partida inesperado dado o prestígio das representações por *embeddings*, é explicável por três fatores concorrentes.

Em primeiro lugar, as mensagens SMS são muito curtas. A representação Word2Vec obtida por média de vetores de palavras é tanto mais representativa quanto maior for o número de tokens da mensagem; em textos muito curtos, a média perde poder discriminativo.

Em segundo lugar, os sinais linguísticos mais fortes de *spam* neste *dataset* são vocabulário específico de alta especificidade ("FREE", "CALL", "WIN", "PRIZE", "£", números de telefone), que TF-IDF identifica diretamente como *features* de alta relevância.

Em terceiro lugar, Word2Vec e Doc2Vec foram treinados apenas no corpus de treino (~4.457 mensagens), que é relativamente pequeno para aprender *embeddings* de qualidade.

No que respeita à TF-IDF face ao BoW, a diferença de desempenho é pequena mas consistente.

4.2. Comparação entre Classificadores

Para representações esparsas (BoW/TF-IDF), sem reamostragem, há 4 combinações comparáveis (2 representações × 2 configurações de pré-processamento). A Regressão Logística vence em 3 destas 4, enquanto o SVM linear vence apenas em TF-IDF+Config A, sendo precisamente aí que se obtém o melhor resultado absoluto do conjunto (F1 = 0,937). A LR é, no geral, o classificador mais consistente nestes espaços esparsos, com tempo de execução muito inferior ao SVM.

Para *embeddings* densos, o padrão não é uniforme entre Word2Vec e Doc2Vec. Em Word2Vec, o *Random Forest* é claramente o melhor (F1 = 0,873). Em Doc2Vec, é o SVM que vence (F1 = 0,863), seguido do *Random Forest*, com a LR a ficar atrás. Em ambos os casos de *embeddings*, a Regressão Logística é o classificador mais fraco - um padrão que se inverte face às representações esparsas, onde a LR é a opção mais consistente.

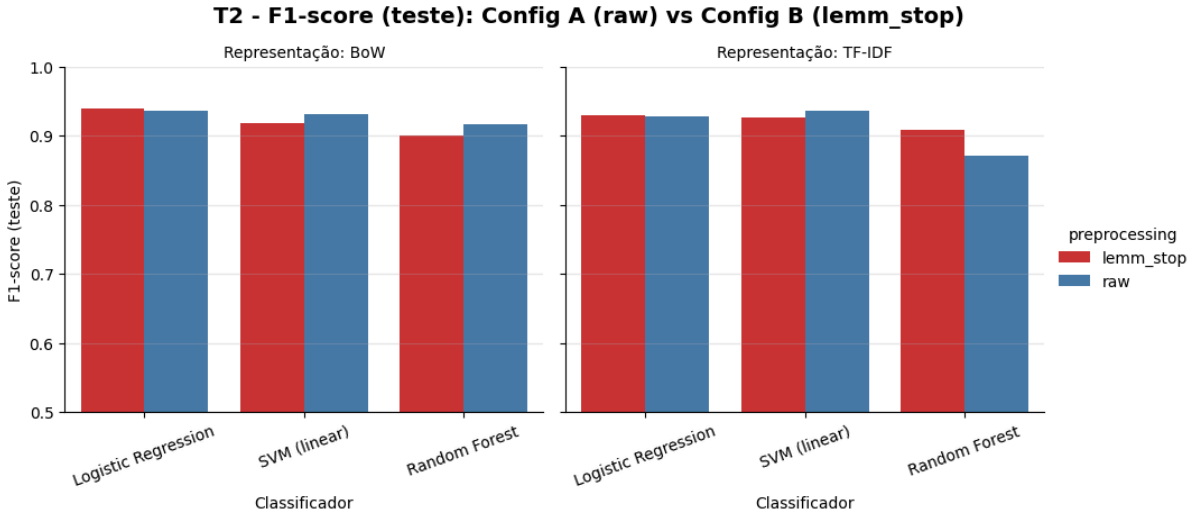
4.3. Análise de *Overfitting*

A diferença entre F1 em validação cruzada e F1 no conjunto de teste é em geral pequena para BoW e TF-IDF, indicando boa generalização. A exceção mais notável é TF-IDF + RF + Config A sem reamostragem (CV F1 = 0,902 vs teste F1 = 0,871, Δ = 0,031), sugerindo alguma tendência para *overfitting* do RF em representações esparsas.

Para *embeddings*, a situação é mais heterogénea: Word2Vec+ LR apresenta a maior divergência do conjunto (CV F1 = 0,721 vs teste F1 = 0,815, Δ = 0,094). Note-se que aqui o teste supera a CV, sugerindo instabilidade no treino dos *embeddings* Word2Vec entre *folds*. Já Doc2Vec + LR segue o padrão inverso e mais habitual, com queda de CV para teste (0,875 → 0,798, Δ = 0,077), mais consistente com *overfitting*.

4.4. Efeito do Pré-processamento (Tarefa T2)

O impacto do pré-processamento é surpreendentemente modesto e inconsistente entre combinações. Para BoW e TF-IDF com LR, ambas as configurações produzem F1 com diferença inferior a 0,005. Para SVM, a Config A supera ligeiramente a Config B em todos os casos. Para RF com TF-IDF, a Config A é evidentemente inferior, sendo este o caso mais relevante em que a lematização ajuda.



A razão pela qual a Config A se mantém competitiva (ou superior) é que a função `clean_text` remove dígitos e símbolos, eliminando parte do vocabulário de *spam* (preços, números de telefone), mesmo antes de qualquer lematização. Adicionalmente, em mensagens SMS, as *stop words* como "call", "free" e "now" têm, neste contexto, elevado valor discriminativo. A sua remoção pela Config B pode prejudicar o desempenho.

4.5. Efeito da Reamostragem (Tarefa T1)

Os resultados da Tabela 4 revelam que a reamostragem não melhora os modelos mais fortes (LR e SVM): o *baseline* sem reamostragem obtém F1 = 0,929 (LR) e 0,937 (SVM), enquanto SMOTE e RUS produzem valores iguais ou inferiores. A reamostragem introduz ruído (SMOTE) ou reduz o tamanho do conjunto de treino (RUS), o que é prejudicial para modelos que já convergem bem com dados desbalanceados.

O único caso em que a reamostragem acrescenta valor é o RF: SMOTE eleva o F1 de 0,871 para 0,929 (+0,058), e RUS para 0,903 (+0,032). O RF é o classificador com menor

capacidade de adaptação a classes desbalanceadas, já que as árvores de decisão tendem a ser enviesadas para a classe maioritária - e o SMOTE corrige exatamente esse enviesamento ao balancear artificialmente as classes no espaço de *features*.

É relevante notar que a reamostragem foi implementada dentro do *pipeline* de validação cruzada (`imblearn.pipeline.Pipeline`), garantindo que os exemplos sintéticos do SMOTE nunca "contaminam" os *folds* de validação.

4.6. Análise de Erros (Tarefa T5)

No melhor modelo (TF-IDF + SVM linear, Config A), foram identificados 16 falsos negativos (*spam* classificado como *ham*) e apenas 1 falso positivo, no conjunto de teste de 1.115 mensagens. O recall da classe *spam* é de 89,3% (133/149).

A análise dos 16 falsos negativos revela padrões recorrentes que explicam a dificuldade de classificação:

- **Destruição de sinais numéricos e monetários.** A função `clean_text` remove todos os dígitos e símbolos não-alfabéticos. Mensagens como "FreeMsg Hey there darling... std chgs to send, £1.50 to rcv" perdem precisamente os tokens mais discriminativos (ex.: valores monetários).
- **Colapso de URLs e números de telefone.** URLs como "http://careers.bloomberg.com" são colapsados num único token ilegível ("httpcareersbloombergcom"); números de telefone como "09099726429" desaparecem por completo.
- **Mensagens muito curtas**. "ringtoneking 84484" é reduzida a "ringtoneking" após remoção do número, um token raro que provavelmente não sobrevive ao filtro `min_df=2` do vetorizador.
- **Erros ortográficos deliberados.** "Low-cost prescripiton drvgs" usa grafia propositadamente alterada ("prescripiton", "drvgs") para evadir filtros baseados em palavras-chave. Sendo variantes raras, são filtradas pelo `min_df`.

O único falso positivo foi "Ok... Ur typical reply...", cuja combinação de abreviatura ("Ur") com pontuação informal pode ter ativado *features* de *spam* aprendidas pelo modelo.

A principal lição desta análise é que a remoção indiscriminada de dígitos e símbolos pela `clean_text` elimina precisamente os sinais mais fortes de *spam* neste dataset. Uma melhoria imediata seria preservar ou normalizar estes tokens antes da remoção.

5. Conclusão

5.1. Principais Conclusões

Este trabalho avaliou sistematicamente 27 combinações (24 das quais são únicas) de representações de texto, classificadores e estratégias de pré-processamento e reamostragem para classificação binária de mensagens SMS.

Os resultados permitem extrair as seguintes conclusões principais:

- As representações esparsas (BoW e TF-IDF) superaram os *embeddings* densos (Word2Vec e Doc2Vec) de forma consistente, coerente com a natureza curta e com vocabulário de *spam* altamente específico das mensagens SMS.
- O melhor modelo obtido foi TF-IDF + SVM linear, com mínima diferença entre validação cruzada e teste (Δ F1 = 0,001), confirmando a sua estabilidade e capacidade de generalização.
- O impacto do pré-processamento é modesto e por vezes contraproducente: a Config A (sem lematização/*stop words*) é, no geral, igual ou superior à Config B, sugerindo que as *stop words* têm valor discriminativo neste contexto específico.
- A reamostragem (SMOTE, RUS) não melhora os modelos lineares já robustos ao desequilíbrio (LR, SVM), mas traz ganhos significativos para o *Random Forest* (SMOTE), o classificador mais sensível ao desequilíbrio de classes.
- A análise de erros revela que a remoção de dígitos e símbolos pela função de limpeza base destrói sinais discriminativos essenciais, sendo a principal fonte de falsos negativos.

5.2. Trabalhos Futuros

Com base nos resultados e na análise de erros, propõem-se as seguintes direções de trabalho futuro:

- Substituir a remoção indiscriminada de dígitos por normalização de padrões específicos (tokenização de preços, URLs e números de telefone), preservando o valor discriminativo destes sinais.
- Avaliar representações BERT (e.g., distilbert-base-uncased) para beneficiar de *embeddings* contextuais pré-treinados em larga escala, potencialmente mais robustos ao vocabulário informal e às abreviações SMS.
- Explorar XGBoost e LightGBM como classificadores adicionais, especialmente em combinação com *embeddings* densos, onde os métodos ensemble baseados em gradient boosting tipicamente superam RF.

6. Referências

- Almeida, T. & Hidalgo, J. (2011). SMS Spam Collection [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84.
- Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011, September). Contributions to the study of SMS spam filtering: new collection and results. In Proceedings of the 11th ACM symposium on Document engineering (pp. 259-262).
- models.doc2vec - Doc2vec paragraph embeddings - gensim. (n.d.). Retrieved June 19, 2026, from https://radimrehurek.com/gensim/models/doc2vec.html
- models.word2vec – Word2vec embeddings - gensim. (n.d.). Retrieved June 19, 2026, from https://radimrehurek.com/gensim/models/word2vec.html
- Padurariu, C., & Breaban, M. E. (2019). Dealing with data imbalance in text classification. Procedia Computer Science, 159, 736–745. https://doi.org/10.1016/j.procs.2019.09.229
- TfidfVectorizer - scikit-learn 1.9.0 documentation. (n.d.). Retrieved June 19, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html